

TP Architecture multi-tiers avec Spring boot

Créer des Api REST avec Spring Boot

Objectif : Nous allons créer un service de paie simple pour gérer les employés d'une entreprise. Les objets employés sont stockés dans une base de données et on y accède via des Api rest.

Configuration :

Pour commencer, vous avez besoin de :

- Un éditeur de texte ou IDE préféré, tel que :
 - [IntelliJ IDEA](#)
 - [VSCode](#)
- [Java 17](#) ou version ultérieure
- Maven 3.5+

Dans ce tutoriel, nous utiliserons Spring Boot. Accédez à Spring Initializr (<https://start.spring.io/>) et ajoutez les dépendances suivantes à un projet :

- Spring Boot DevTools
- Spring Web
- Spring Data JPA
- MySQL Driver

Comme nom de projet, choisir « **Paie** », puis choisissez « Générer le projet ». Un fichier **.zip** se télécharge. Décompressez-le. Vous y trouverez un projet simple, basé sur Maven, incluant un fichier pom.xml.

Fichier **application.properties**

```
spring.application.name=miage2025v1
server.port= 8085
spring.datasource.url=jdbc:mysql://localhost:3306/paie-v1
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
```

Domaine

Le code suivant définit un Employee dans notre système.

```
package pae;
import java.util.Objects;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;

@Entity
class Employee {

    private @Id
    @GeneratedValue Long id;
    private String name;
    private String role;

    Employee() {}

    Employee(String name, String role) {
        this.name = name;
        this.role = role;
    }

    public Long getId() {
        return this.id;
    }

    public String getName() {
        return this.name;
    }

    public String getRole() {
        return this.role;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setRole(String role) {
        this.role = role;
    }
}
```

Bien que petite, cette classe Java contient beaucoup :

- `@Entity` est une annotation JPA permettant de préparer cet objet au stockage dans un magasin de données basé sur JPA.
- `id`, `name`, et `role` sont des attributs de notre classe `Employee` comme domaine. `id` est marqué avec plus d'annotations JPA pour indiquer qu'il s'agit de la clé primaire et est automatiquement renseigné par le JPA.

Avec cette définition d'objet de domaine, nous pouvons maintenant nous tourner vers Spring Data JPA pour gérer les interactions avec la base de données.

Repository

```
package paie;

import org.springframework.data.jpa.repository.JpaRepository;

interface EmployeeRepository extends JpaRepository<Employee, Long> {

}
```

Spring Boot simplifie l'accès aux données. En déclarant EmployeeRepository de type interface, nous pouvons automatiquement :

- Créer de nouveaux employés
- Mettre à jour les employés existants
- Supprimer les employés
- Rechercher des employés (un, tous ou rechercher par propriétés simples ou complexes)

Pour obtenir toutes ces fonctionnalités gratuites, il suffit de déclarer une interface qui étend Spring Data JPA JpaRepository, en spécifiant le type de domaine Employee et id type Long.

Controller

```
package paie;

import java.util.List;

import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
class EmployeeController {

    private final EmployeeRepository repository;

    EmployeeController(EmployeeRepository repository) {
        this.repository = repository;
    }

    @GetMapping("/employees")
    List<Employee> all() {
        return repository.findAll();
    }

    @PostMapping("/employees")
    Employee newEmployee(@RequestBody Employee newEmployee) {
        return repository.save(newEmployee);
    }

    @GetMapping("/employees/{id}")
    Employee one(@PathVariable Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new EmployeeNotFoundException(id));
    }

    @PutMapping("/employees/{id}")
    Employee replaceEmployee(@RequestBody Employee newEmployee, @PathVariable Long id) {
        return repository.findById(id)
            .map(employee -> {
                employee.setName(newEmployee.getName());
                employee.setRole(newEmployee.getRole());
                return repository.save(employee);
            })
            .orElseGet(() -> {
                return repository.save(newEmployee);
            });
    }

    @DeleteMapping("/employees/{id}")
    void deleteEmployee(@PathVariable Long id) {
        repository.deleteById(id);
    }
}
```

- L'annotation `@RestController` indique que la classe est un contrôleur et que les méthodes renvoient directement des données (généralement en JSON ou XML), et non une vue (HTML, JSP...).
- Un `Employee Repository` est injecté par le constructeur dans le contrôleur.
- Nous avons des routes pour chaque opération (`@GetMapping`, `@PostMapping`, `@PutMapping` et `@DeleteMapping`, correspondant aux appels HTTP GET, POST, PUT, et DELETE).

Lancer le projet

```
mvn spring-boot:run
```